Cours d’Algorithmique et Méthodes de Programmation

Prof. Dr. Merlec MPYANA

Sciences et Ingénierie informatiques
(Génie Logiciel)

Mai 2024

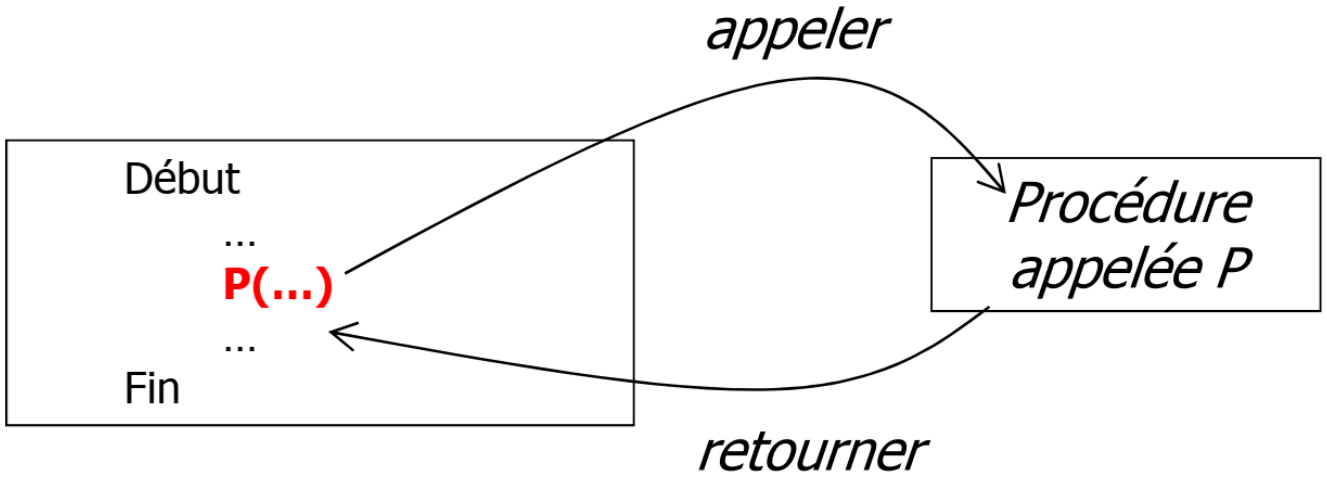
Procédures et Fonctions

Procédures

- **Une procédure est un sous-algorithme qu'on écrit à part.**
  - **Une procédure peut être utilisée depuis l'algorithme principal, on dit alors qu'elle est appelée.**
    - *Pour appeler une procédure, on utilise le nom_de_ la_procedure().*
  - **Une procédure commence par le mot-clé *Procédure*, suivi du nom qu'on don ne à la procédure, puis des éventuels arguments de la procédure.**
    - *La procédure peut posséder ses propres variables internes (variable locales).*
    - *Le corps d'une procédure commence par le mot-clé Début, et se termine par Fin*

Procédures

- ***Remarques***
  - **Un argument est une valeur ou une variable qu'on transmet à la procédure, depuis l'algorithme principal, lors de son appel.**
  - **Une procédure peut appeler une autre procédure.**



Procédures

❑ Exemple

- **La procédure suivante génère et affiche un nombre aléatoire.**
- **Le nom de la variable (*borneSup*) qui sert d'argument lors de l'appel de la procédure, n'a pas besoin d'être le même que le nom de la variable interne à la procédure (*borne*).**

```
Procédure afficheAleatoire(borne : nombre)
    nombreAlea ← alea(1, borne)
    Écrire "Le nombre généré est ", chaine(nombreAlea), "."
Fin
```

L'algorithme principal qui appelle cette fonction est le suivant.

```
Variable combienNombres, compteur, borneSup : nombre
Début
    Écrire "Veuillez choisir combien de nombres seront générés."
    Lire combienNombres
    Écrire "Les nombres varieront de 1 à une borne supérieure."
    Écrire "Quelle est la valeur de la borne supérieure ?"
    Lire borneSup
    Pour compteur De 1 À combienNombres
        Appeler afficheAleatoire(borneSup)
    FinPour
```

Fonctions

- **Une fonction est un enchaînement d'instruction (sous-programme) auquel on donne un nom pour pouvoir le réutiliser plus tard.**
  - **Toute fonction résous un problème précis, et est donc accompagnée de la spécification de ce problème.**
  - **Une fonction commence par le mot-clé *Fonction*, suivi du nom qu'on donne à la fonc tion, puis des éventuels arguments de la fonction.**

- *Pour appeler une fonction, on utilise le nom_de_ la_fonction().*
- *Une fonction peut posséder ses propres variables internes (variable locales).*
- *Le corps d'une fonction commence par le mot-clé Début, et se termine par Fin*

Procédures et Fonctions

- **Les procédures sont assez proches des fonctions.**
- **Une différence d'usage importante est que :**
  - **Les fonctions renvoient une valeur, qui sera affectée à une variable ;**
  - **Les procédures contient une série d'instructions,**
    - *un véritable morceau de programme, avec des instructions de lecture, écriture, des structures conditionnelles, des boucles, etc.*
    - *Elle permettent de découper l'algorithme principal en plusieurs sous-algorithmes et de le rendre plus lisible.*

Procédures et Fonctions

❑ Remarques

- **Cette distinction vaut pour l'algorithmique, mais elle n'est pas valable pour tous les langages informatiques.**
  - Dans certains cas, fonctions et procédures sont synonymes.
- **Les fonctions et les procédures participent aussi à la factorisation du code, comme les boucles.**
- **Il est parfois possible de faire la même chose avec une fonction ou avec une procédure.**

Procédures et Fonctions

- **Dans une procédure, il est possible de modifier les valeurs des variables de l'algorithme principal.**
  - **Il faut pour cela passer les variables qu'on veut modifier comme arguments.**
  - **Il faut de plus indiquer qu'on les passe par référence avec le mot-clé *Ref*.**

Les deux modes de passage

- **Passage par valeur (défaut):**
  - **La procédure appelée dispose d'une copie de la valeur;**
    - **elle peut la modifier, l'information initiale dans l'appelante n'est pas affectée par ces modifications.**
- **Passage par référence**
  - **La procédure appelée modifie directement la variable en utilisant sa référence.**
    - **Il faut de plus indiquer qu'on les passe par référence avec le mot-clé *Ref*.**

Exemple : Passage par valeur (défaut):

Procédure PermuterMarchePas **(**x , y **: Entier par valeur)**
Variables tmp **: Entier**
Début
    tmp ← x
    x ← y
    y ← tmp
Fin

Programme Test
Variables a, b **: Entier**
Début
    a ← 1
    b ← 5
    PermuterMarchePas(a,b)
    Ecrire ("a = ", a, ", b = ", b)
Fin

Exemple : Passage par référence

Procédure PermuterMarchePas **(**x , y **: Entier par référence)**
Variables tmp **: Entier**
Début
    tmp ← x
    x ← y
    y ← tmp
Fin

Programme Test
Variables a, b **: Entier**
Début
    a ← 1
    b ← 5
    PermuterMarchePas(a,b)
    Ecrire ("a = ", a, ", b = ", b)
Fin

Relation entre type et mode de passage

❑ Relation entre type et mode de passage

	Passage par valeur	Passage par référence
Paramètre d’entrée	oui	oui
Paramètre de sortie	non	oui

Exemple: Argument passé par référence

- **Voici ci-dessous un programme de jeu. Le joueur dispose d'une réserve d'argent et il doit parier sur un nombre, entre 1 et 10, en misant une somme d'argent.**

  - **Une procédure gère toute la partie tirage de nombre aléatoire et calcul du résultat.**
  - **Les arguments *argent* et *partieGagnee* sont passés par référence à la procédure, qui peut donc directement les modifier.**

Algorithme du Programme Principal

```
Algorithme JeuTirageNombreAléatoire
Variable argent, partieGagnee, nombrePari, mise : nombre
Variable continuer : chaine
Début
    continuer ← "oui"
    argent ← 100
    partieGagnee ← 0
    Écrire ("Vous avez ", chaine(argent), " $ pour jouer.")
    TantQue continuer = "oui"
        Écrire ("Sur quel nombre pariez-vous (entre 1 et 10) ?")
        Lire pari
        Écrire ("Combien misez-vous ?")
        Lire mise
        Appeler lancerPartie(pari, mise, argent Ref, partieGagnee Ref)
        Écrire ("Voulez-vous continuer à jouer ? (oui/non)")
        Lire (continuer)
    FinTantQue
Fin
```

Algo de la procédure appelée *lancerPartie*

```
Variable nombreAleatoire : nombre
Procédure lancerPartie(pari, mise, argent Ref, partieGagnee Ref : nombre)
    nombreAleatoire ← alea(1,10)
    Si nombreAleatoire = pari
        partieGagnee ← partieGagnee + 1
        argent ← argent + mise * 5
        Écrire (“Vous avez gagné !”)
    Sinon
        argent ← argent – mise
        Écrire (“Vous avez perdu !”)
    FinSi
    Écrire ("Vous avez ", chaine(argent), “ $ pour jouer.”)
Fin
```

Récursivité

Récursivité

- Un algorithme est récursif quand dans l'enchaînement des instructions qui le composent, l'une de celles-ci est un appel à ce même algorithme.

- Algorithmes récursifs: dans l'algorithmique et la programmation

- Il y a plusieurs catégories d'algorithmes récursifs en considérant.
  - Le mode d'appel : *direct/indirect*
  - Le nombre de paramètres sur lesquels porte la récursion : *arité*
  - Le nombre d'appels récursifs : *ordre de récursion*
  - Le genre de retour : *terminal/non terminal*

1. Récursivité Direct et Indirect

- Un algorithme récursif qui s’appelle lui-même dans son corps est dit direct.
  - L'exemple de la factorielle est trivial.

- Un algorithme récursif est dit indirect s'il est défini par un ensemble de fonctions qui s'appellent en chaîne.

1. Récursivité Direct et Indirect

❑ Algorithme récursif indirect pour déterminer la parité d'un entier.

- Pour construire une définition récursive de la parité, remarquons tout d'abord que 0 est un entier pair, et qu'il n'est pas impair. Ensuite remarquons qu'un entier n est pair (respective ment impair) si l'entier n-1 est impair (respectivement pair).

```
Entrées :  n un entier
Sorties :  vrai si n est pair, faux si n est impair

fonction Pair(n)
début
    si n=0 alors
        retourner vrai
    sinon
        retourner Impair(n-1)
fin
```

```
Entrées :  n un entier
Sorties :  vrai si n est impair, faux si n est pair

fonction Impair(n)
début
    si n=0 alors
        retourner faux
    sinon
        retourner Pair(n-1)
fin
```

1. Récursivité Direct et Indirect

❑ Algorithme récursif indirect pour déterminer la parité d'un entier.

- Pour construire une définition récursive de la parité, remarquons tout d'abord que 0 est un entier pair, et qu'il n'est pas impair. Ensuite remarquons qu'un entier n est pair (respective ment impair) si l'entier n-1 est impair (respectivement pair).

- Si l'on trace Pair(3) on obtiendra :

$$
\begin{array}{ccccccc}
Pair(3) & \leftarrow & \underline{\text{Faux}} & & & & \\
 & \searrow & \uparrow & & & & \\
 & & Impair(2) & \leftarrow & \underline{\text{Faux}} & & \\
 & & & \searrow & \uparrow & & \\
 & & & & Pair(1) & \leftarrow & \underline{\text{Faux}} \\
 & & & & & \searrow & \uparrow \\
 & & & & & & Impair(0)
\end{array}
$$

2. Récursivité Arité

- L'arité d'un algorithme est le nombre de paramètres d'entrée.
  - Un algorithme récursif peut effectuer des appels récursifs en modifiant un nombre quel conque non nul de ses paramètres.
  - Dans l'exemple de la fonction factorielle, l'algorithme prend un paramètre d'entrée et le modifie lors des appels récursifs.
  - Dans le cas de la puissance entière (proposé en exercice), l'algorithme prend deux entr ées, la base et l'exposant. Dans les appels récursifs seul l'exposant est modifié.

2. Récursivité Arité

- Voici un exemple de récursion sur deux paramètres.
  - Algorithme récursif de calcul du PGCD de deux entiers

Entrées : a, b entiers
Résultat : le plus grand commun diviseur de a et b s'il existe, 0 sinon

```
fonction PGCD(a,b)
début
    si a=0 ou b=0 alors
        retourner 0
    sinon si a=b alors
        retourner a
    sinon si a > b alors
        retourner PGCD(a-b,b)
    sinon
        retourner PGCD(a,b-a)
fin
```

2. Récursivité Arité

- **Exemple : définition récursive de la factorielle**

$$n! = \begin{cases} 1 & \text{si } n = 0 \\ n(n-1)! & \text{si } n > 0 \end{cases} \quad n \in \mathbb{N}$$

- **Les deux caractéristiques principales d'une définition récursive intéressante :**
  - *Présence d'un cas particulier ne faisant pas un appel à la définition, mais donnant un résultat*

$$n! = 1 \text{si } n = 0$$

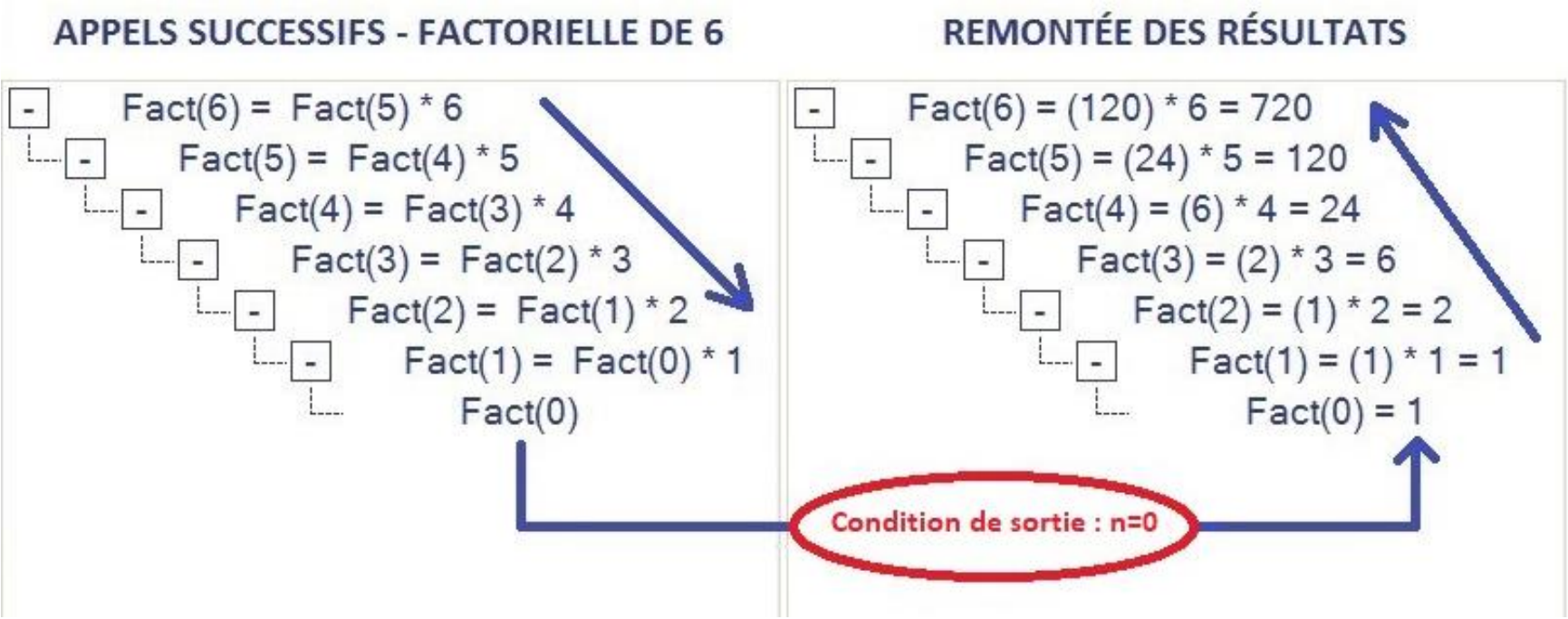
  - *Présence d'un appel récursif mais avec un paramètre plus petit* $n! = n(n-1)! \text{ si } n > 0$

Utiliser une définition récursive est souvent considéré comme élégant, concis et lisible.

2. Récursivité Arité

❑ Exemple d'algorithme de la factorielle

- La fonction génère des appels récursifs successifs pour *Fact*(n), *Fact*(*n*-1),..,*Fact*(0), puis remonte les résultats à partir de *Fact*(0)=1 (condition de sortie), puis *Fact*(1)=*Fact*(0)*1=1 *1,.., *Fact*(*n*)=*Fact*(*n*-1)**n*, suivant le schéma :



2. Récursivité Arité

❑ Exemple d'algorithme de la factorielle

- Nous retrouvons en ligne 4 la condition d'arrêt : aucun appel récursif n'est mis en œuvre, et en ligne 6 l'appel récursif.

```
Algorithme : Factorielle
  Entrées :  n un entier
  Sorties :  n! la factorielle de n

1 fonction Fact(n)
2 début
3 |   si n = 0 alors
4 |   |   retourner 1
5 |   sinon
6 |   |   retourner n * Fact(n-1)
7 fin
```

3. Ordre de récursion

- Jusqu'à présent, tous les algorithmes récursifs évoqués ne font qu'un appel récursif à la fois.
  - Cad que pour déterminer la valeur d'un appel, on n'a besoin que d'un appel récursif.
  - Ces algorithmes sont dits d'ordre 1.

- Certains algorithmes font plusieurs appels récursifs, comme la version naïve du calcul de la suite de Fibonnaci.
  - Celle-ci doit faire deux appels récursifs, une fois avec le paramètre n-1 et une seconde fois avec le paramètre n-2 pour calculer une valeur de retour.
  - C’est un algorithme récursif d'ordre 2.

3. Ordre de récursion

- **Algorithme récursif pour calculer le nième terme de la suite de Fibbonacci.**

La suite de Fibbonacci est définie par $(F_n)_{n\in\mathbb{N}}$: $\begin{cases} F_0 = 1 \\ F_1 = 1 \\ F_{n+2} = F_{n+1} + F_n, \forall n \in \mathbb{N} \end{cases}$

Entrées : n entier, indice du terme de la suite de Fibbonacci
Résultat : le terme de rang n de la suite de Fibbonacci

```
fonction Fibb(n)
début
    si n=0 alors
        retourner 1
    sinon si n=1 alors
        retourner 1
    sinon
        retourner Fibb(n-1) + Fibb(n-2)
fin
```

4. Récursion terminale

- Pour déterminer si un algorithme est terminal, il faut regarder comment il génère les valeurs de retour.
  - Si ce sont soit des constantes, soit des valeurs directement issue d'un appel récursif, sans aucune autre modification, alors l'algorithme est dit terminal.
  - Dans le cas contraire, il sera dit non-terminal.
- Dans les exemples précédents, factorielle est non terminal,
  - Car, bien que le cas d'arrêt renvoie une constante, celui de l'appel récursif quant à lui modifie la valeur renvoyée en la multipliant par *n* avant de lui-même renvoyer une valeur.

Fonction Récursive

- **Une fonction récursive est une fonction qui s'appelle elle-même.**
  - **Chaque appel à la fonction est indépendant des autres, avec ses propres variables.**
    - *Une récursion a toujours la forme suivante :*

Si (*cas simple*) **Alors**
(*solution immédiate*)
Sinon
(*solution récursive, impliquant un cas plus simple que le problème original*)
FinSi

Fonction Récursive

❑ Exemple: Puissance entière d'un entier

- **La fonction puissance entière peut être définie ainsi :**

$$x^n = \begin{cases} 1 & \text{si } n = 0 \\ x.\,x^{(n-1)} & \text{si } n > 0 \end{cases} \qquad x, n \in \mathbb{N}$$

1. Cette définition respecte-t-elle les caractéristiques d'une définition récursive intéressante ?

2. Écrivez l'algorithme de la fonction *ipower*(*x*,*n*) correspondante.

Utiliser une définition récursive est souvent considéré comme élégant, concis et lisible.

Fonction Récursive

❑ Exemple: Puissance entière d'un entier

- **La fonction puissance entière peut être définie ainsi :**

$$x^n = \begin{cases} 1 & \text{si } n = 0 \\ x.\,x^{(n-1)} & \text{si } n > 0 \end{cases} \qquad x, n \in \mathbb{N}$$

1. Cette définition respecte-t-elle les caractéristiques d'une définition récursive intéressante ?

2. Écrivez l'algorithme de la fonction ***ipower(x,n)*** correspondante.

Utiliser une définition récursive est souvent considéré comme élégant, concis et lisible.

Structure de donnée récursive

- **Si nous appliquons un schéma récursif à la création de structures de données, nous créons une structure de données récursive.**

- **Une structure de donnée *S* est dite récursive**
  - **si l'une de ses composantes est aussi une structure de donnée *S* ou si l'un de ses compo santes est une structure de donnée faisant appel à S.**
  - **Ces structures deviennent nécessaires dès que l'on a besoin de les agrandir dynamiquement.**
    - *L'exemple le plus simple est sans doute celui de la liste. On peut implémenter une liste d'objets avec une structure élémentaire comme un tableau.*
    - *Mais la gestion dynamique d'un tableau peut-être difficile à maintenir.*

Structure de donnée récursive

- **Une liste peut alors être considérée sous une forme récursive.**
  - **une liste est composée de (1) un élément de la liste et (2) une sous-liste d'autres éléments**

Exemple

Supposons que nous disposions de la liste de personnes suivante :

Alice, Bob, Charles, Damien

Une manière de représenter cette liste serait un tableau :

Alice	Bob	Charles	Damien	(vide)	(vide)

Un tableau possédant une certaine longueur, il a ici été surdimensionné (2 cases sont vides)

Une autre manière de représenter cette liste serait une liste simplement chaînée :

$$\left(Alice, \circ \mapsto \left(Bob, \circ \mapsto \left(Charles, \circ \mapsto \left(Damien, \circ \mapsto ()\right)\right)\right)\right)$$

Q & R

Contact: mlec.academia@gmail.com